Tomato Leaf Decision Support

Final App Review and Technical Walkthrough

Disease diagnosis, context-aware guidance, and treatment follow-up

IMAGE | MODEL | CONTEXT | ACTION

Prepared for the final project review

Issac Jacob Abraham | Naman Sharma

29 October 2026

1. App overview

This web app combines the existing MobileNetV2 tomato-leaf classifier with a transparent decision-support layer. It accepts a leaf photograph, checks whether the model result is sufficiently clear, estimates visible discoloration, takes optional crop and weather context, and presents general management options. A farmer can later record an outcome for follow-up guidance.

The result is a first-line triage prototype. The disease classifier is trained; the severity and risk components are simple rules and image-color heuristics. The app does not diagnose with certainty, prescribe a locally approved pesticide, or learn from feedback.

End-to-end workflow



The uncertainty gate can stop the flow after image classification. When it does, the app asks for another photo and withholds treatment recommendations until a more confident result is available.

Main code modules

File	Role in the app
app.py	Streamlit page, image upload, inference, uncertainty gate, input forms, and results.
train.py	MobileNetV2 construction, two-phase training, augmentation, checkpoints, and training curves.
context_engine.py	Visual severity proxy, risk rules, contextual priority, follow-up guidance, and local feedback log.
treatment_recommendations.py	Disease descriptions and class-specific treatment/prevention reference text.
evaluate.py and data_prep.py	Dataset split/preparation and held-out validation metrics/confusion matrix.

2. Image diagnosis and severity

The uploaded image is converted to RGB, resized to 224 by 224 pixels, and passed through MobileNetV2 preprocessing. The saved model returns a softmax score for each of ten classes. The app displays the highest scoring class and its score as confidence.

The classifier covers these PlantVillage tomato labels

Class	Class
Bacterial spot	Spider mites Two spotted spider mite
Early blight	Target Spot
Late blight	YellowLeaf Curl Virus
Leaf Mold	mosaic virus
Septoria leaf spot	healthy

Uncertainty-aware behavior

- The result is treated as uncertain if the top class is below 70% confidence, or if its lead over the second class is less than 12 percentage points.
- For an uncertain image, the app requests a new in-focus photo in natural light and does not issue the treatment section.

These are configurable prototype thresholds. Softmax confidence is not calibrated probability that the diagnosis is correct.

Visible severity estimate

A basic RGB color heuristic finds green-ish pixels, estimates the fraction that look less green, and maps it to Low (under 12%), Moderate (12% to under 30%), or High (30% or more). It requires at least 250 candidate pixels. The user can adjust the level before it feeds the context rules.

This is not a learned severity model or validated lesion segmentation. Background, lighting, leaf angle, cultivar, nutrient stress, and camera processing can all change the estimate. If too few green-ish pixels are found, severity is marked unavailable.

3. Crop context and progression-risk rules

Optional environmental measurements are left out of scoring unless the user marks them as available. Crop stage and prior treatment response are also collected. The app shows how much of the environmental context was supplied.

Inputs used

Input	How it is used
Temperature	For selected fungal/bacterial disease groups, 15 to 25 C adds a heuristic risk contribution.
Relative humidity	80% or higher adds risk for the moisture-associated disease group; below 45% adds a spider-mite rule.
Leaf wetness	8 hours/day or more adds a moisture-associated risk contribution.
Rainfall	10 mm or more in the last 24 hours adds a moisture-associated risk contribution.
Soil moisture	Below 20% adds a water-stress note, but does not add points to the risk score.
Crop stage and history	Crop stage appears in the explanation. A previously ineffective treatment changes the displayed priority to reassess; an effective history is captured but does not currently change the rule score.

Risk score logic

- Non-healthy predictions start at 15 points. High severity adds 25; Moderate adds 12; Low or unavailable adds no severity points.
- For blight, spot, mold, and related classes: high humidity adds 15, long leaf wetness adds 20, recent rainfall adds 10, and the broad 15 to 25 C range adds 8.
- For spider mites, humidity below 45% adds 12. Low soil moisture contributes a note only. The score is capped at 100.
- Bands are Low below 35, Moderate from 35 to 64, and High at 65 or above. Healthy predictions return a score of zero.

The score is a transparent heuristic indicator, not a probability, trained predictor, or validated disease forecast. If no weather values are supplied, the display reports 0/5 environmental inputs and the score mostly reflects the base rule and selected severity.

4. Treatment recommendations and follow-up

For confident disease results, a lookup table provides a plain-language class description, general treatment reference, and prevention text. The context engine adds an action priority and presents cultural, biological, and chemical management categories. Chemical options are framed as general guidance and explicitly direct the user to local registration, product label, protective equipment, pre-harvest interval, and extension-officer confirmation.

How context changes the recommendation

- High rule-based risk prioritizes prompt scouting and confirmation.
- Low estimated severity favors preventive cultural measures and close monitoring.
- Other cases favor integrated management with local confirmation.
- A previously ineffective treatment changes the priority to reassess the diagnosis and avoid repeating treatment without expert advice.

Closed-loop feedback

After treatment, the user can enter follow-up severity, a farmer-reported outcome (improved, no change, or worsened), and an optional description of what was applied. The app records one JSON line in model/treatment_feedback.jsonl with a timestamp and gives follow-up guidance based on the reported response and severity change.

The feedback is a local record only. It does not change model weights, retrain the risk rules, or establish that a treatment caused the reported outcome. Comparable follow-up photos should use similar lighting, distance, and leaf orientation.

Safety and scope

The system is a first-line educational and triage aid, not a replacement for an agronomist or local extension officer. A classifier can confuse visually similar problems; the app should not be used as the sole basis for applying chemicals or removing crops.

5. Model training and reported evaluation

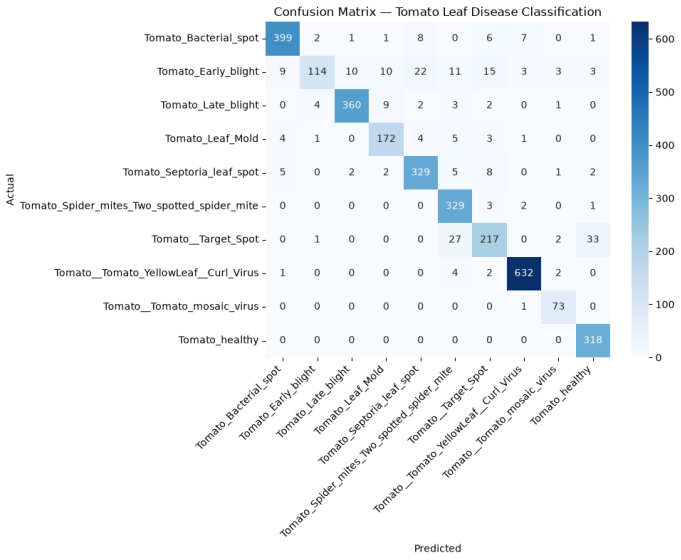
The classifier uses ImageNet-pretrained MobileNetV2 as a feature backbone, a global average pooling layer, dropout, a 128-unit dense layer, and a ten-class softmax output. Training first freezes the backbone, then fine-tunes the final 30 backbone layers at a low learning rate. Training augmentation includes rotations, shifts, shear, zoom, and horizontal flips.

The evaluation files included with the trained model report the following validation-split results

Metric	Reported value
Accuracy	92.03%
Macro precision	91.03%
Macro recall	90.18%
Macro F1-score	90.04%

These values are copied from model/eval_metrics.json. The original project describes an 80/20 class-wise train/validation split. The supplied package does not include the raw dataset or a test-set report, so these numbers should be presented as validation results, not as field performance or external test accuracy.

Validation confusion matrix



Use the matrix to discuss class-specific errors; the aggregate metrics can hide disease pairs that are visually similar.

6. Running the app and final review

The delivered project includes the trained H5 model, class mapping, evaluation metrics, confusion matrix, and training curves. It does not include the training dataset. The Windows shortcut starts a batch launcher, creates a local virtual environment, installs dependencies on first run, and launches Streamlit in a browser.

User flow for a live demo

- Double-click Tomato Leaf App in the project folder. Keep the launcher window open while the app is running.
- Upload a clear, well-lit JPG or PNG of a single tomato leaf. A close view with the leaf filling most of the frame is preferred.
- If confidence is low or the top two classes are close, show the request for a second photo. No treatment options are issued for that result.
- For a confident result, review the severity estimate, optionally adjust it, and enter only environmental values that are actually available.
- Walk through the risk explanation, general treatment categories, label caution, and post-treatment feedback form.

Setup note

The project requirements were adjusted for Python 3.14 using TensorFlow 2.22.0rc0 or newer. TensorFlow 2.22.0rc0 provides a Windows CPython 3.14 wheel, but it is a release candidate. The saved model was created in an older TensorFlow environment; its compatibility must be confirmed on the computer by successfully loading the model and running a prediction.

Limitations and next research steps

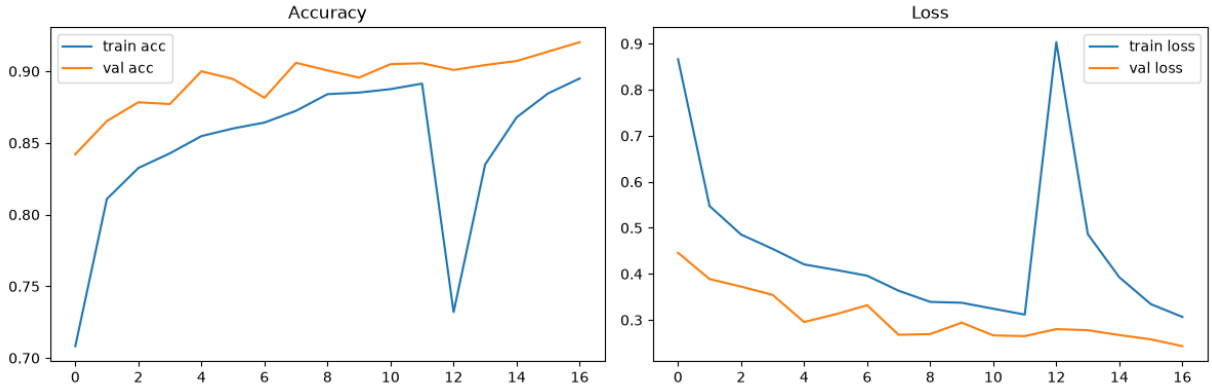
- PlantVillage images have controlled backgrounds; field performance needs evaluation on independently collected field photos.
- Severity and progression risk need expert labels and field measurements before they can be described as validated estimators.
- Treatment feedback needs structured, repeated observations and a comparison design before treatment effectiveness can be inferred.
- A stronger study should report a held-out test set, per-class precision/recall, calibration, abstention coverage, and field validation.

Final review conclusion

The app now demonstrates a complete user-facing workflow from leaf-image classification through context-aware guidance and follow-up logging. Its strongest contribution is an explainable decision-support layer around an existing classifier. The classifier metrics are useful benchmark results; the severity, risk, and treatment adaptation components remain prototypes that need field evidence before performance claims can be made.

Appendix. Training history and delivered artifacts

The training script saves the accuracy and loss curves from the frozen-backbone phase and fine-tuning phase. The chart below is included with the trained model package and can support a discussion of convergence and overfitting. The full raw dataset is not part of the package, so reproducing training requires obtaining PlantVillage images and running data_prep.py first.



Training history provided in model/training_curves.png.

Artifacts in the delivered model folder

- tomato_disease_model.h5 - trained classifier used by the app.
- class_indices.json - class names and output indices used to interpret predictions.
- eval_metrics.json and confusion_matrix.png - validation summary and per-class error view.
- training_curves.png - training/validation accuracy and loss history.
- treatment_feedback.jsonl - created locally after a user saves follow-up feedback; not part of the original trained model.